



Programovacie jazyky

Racket, lenivé vyhodnocovanie, makrá

RNDr. Richard Ostertág, PhD. (ostertag@dcs.fmph.uniba.sk)

10. mája 2018

Katedra informatiky, Fakulta matematiky, fyziky a informatiky
Univerzita Komenského, Bratislava

Programovací jazyk Racket

- funkcionálny jazyk, vychádzajúci z jazyka Scheme
 - lambda funkcie, uzávery, ...
 - ale má aj imperatívne vlastnosti
 - meniteľné premenné
 - funkcie s vedľajšími efektami
- používa dynamický typový systém
 - počas kompilácie sa nerobí statická typová kontrola
 - kompilátor akceptuje viac programov
 - za cenu, že niektoré chyby objavajú až počas behu
- má striktné vyhodnocovanie argumentov
- minimalistická syntax
- makrá
- a iné, ktoré si nebudeme spomínať
 - moduly, continuations, kontrakty, ...

Inštalácia vývojového prostredia DrRacket

- Hlavná stránka jazyka: <http://racket-lang.org/>
- Vývojové prostredie DrRacket:
<http://download.racket-lang.org/>
 - distribúcia: Racket
 - platforma: Windows (x64, 64-bit)
- po inštalácii spustiť program DrRacket
 - uistite sa, že vľavo dole je vybrané:
„Determine language from source“
- Dokumentácia: <http://docs.racket-lang.org/guide/index.html>
- prostredie DrRacket pomáha učeniu
 - integrovaná nápoveda
 - skúste ukázať myšou na premennú v okne definícií
 - sledovanie rozbaľovania makier

Používanie prostredia DrRacket

- program sa píše v hornom okne definícií (definitions window)
 - Ctrl+D – skryje/ukáže toto okno
 - prvý riadok programu musí byť:
`#lang racket` ;aby sa zvolil správny jazyk
(DrRacket podporuje viac jazykov)
 - Tab – opraví zarovnanie riadku (výberu)
- po stlačení tlačidla Run (alebo Ctrl-R)
 - REPL spracuje definície z horného okna
 - všetky staré definície sa zabudnú
 - REPL je spodné okno interakcií (interactions window)
 - Ctrl+E – skryje/ukáže toto okno
 - Alt+P/N – predošlý/nasledujúci príkaz z histórie REPL
- program je možné predčasne zastaviť stlačením tlačidla Stop

Základná syntax jazyka Racket

- Program pozostáva z termov.
- Term je:
 - atóm – napríklad #t, #f, 1, 2.0, null, "hello", x, ...
 - špeciálna forma – napríklad define, lambda, if
 - makrá nám neskôr umožnia definovať svoje vlastné špeciálne formy
 - postupnosť – niekoľko termov oddelených medzerou v zátvorkách (t_1 t_2 ... t_n)
 - ak t_1 je špeciálna forma, tak sémantika postupnosti je špeciálna
 - inak ide o volanie funkcie t_1 s argumentami $t_2, \dots, t_n$

Poznámka

Namiesto normálnych (...) je kedykoľvek možné použiť hranaté [...].

Pri písaní) ju DrRacket nahradí za], ak jej zodpovedá [.

Definovanie premenných a funkcií

- definovanie premennej:
 - (**define** x 42) – priradí 42 do premennej x
 - (**define** x (+ (* 4 10) 2)) – volanie funkcie, vždy v prefixovom plne uzátvorkovanom tvare ($f\ a_1\ a_2\ \dots\ a_n$)
- definovanie funkcie:
 - (**define** cube (**lambda** (x) (* x x x)))
 - (cube 10) $\rightsquigarrow$ 1000
 - (**define** (cube x) (* x x x)) – syntaktický cukor
 - Racket má skutočné viac-argumentové funkcie:
 - (**define** (objem a v) (* a a v))
 - (objem 10 3) $\rightsquigarrow$ 300
 - (objem 10) $\rightsquigarrow$ objem: arity mismatch
 - aj funkcie s premenlivým počtom argumentov:
 - (**define** cnt-args (**lambda** xs (**length** xs)))
 - (cnt-args "ahoj" 1) $\rightsquigarrow$ 2

Currying

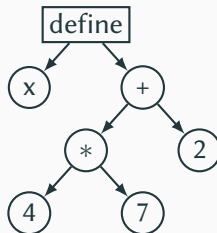
```
1 (define objem  
2   (lambda (a)  
3     (lambda (v)  
4       (* a a v)))))
```

- `(objem 10)` $\rightsquigarrow$ `#<procedure>`
- `(define f (objem 10))`
 - `(f 3)` $\rightsquigarrow$ 300
- `((objem 10) 3)` $\rightsquigarrow$ 300
- `(objem 10 3)` $\rightsquigarrow$ `objem: arity mismatch`

Aké výhody prináša totálne zátvorkovanie?

- konverzia zdrojového kódu do abstraktného syntaktického stromu je triviálna (parsovanie)

```
1 (define
2   x
3   (+
4     (*
5       4
6       7)
7     2))
```



- netreba riešiť prioritu operátorov
- rozdelenie do riadkov a odsadenie je voliteľné a ľahko automatizovateľné
- medzi programom a zoznamom nie je syntaktický rozdiel

Podobný princíp využíva napríklad XML (napríklad HTML).

Na zátvorkách vždy záleží

- nikdy nie sú voliteľné
- nikdy ich nemôžeme len tak pridať/vynechať
 - pre zvýšenie čitateľnosti
- $(\star (+ 1 2) (- 3 1)) \rightsquigarrow 6$
- $(\star ((+ 1 2)) (- 3 1)) \rightsquigarrow$ application: not a procedure
 - (3) je volanie funkcie 3 bez argumentov
(nie číslo v zbytočných zátvorkách)
- $(\star ((+) 1 2) (- 3 1)) \rightsquigarrow$ application: not a procedure
 - $(+)$ je volanie funkcie $+$ bez argumentov, vracia 0
následne $(0 1 2)$ je volanie funkcie 0 s argumentami 1 a 2
- bez statickej typovej kontroly sa chýbajúce/nadbytočné zátvorky prejavajú až za behu

Riadiace konštrukcie – if

(**if** podmienka true-výraz false-výraz)

- podmienka – testovací výraz
- true-výraz – vráti sa, ak sa podmienka $\neq$ **#f**
 - v staticky typovanom jazyku by bolo = **#t**
 - považuje za **#t** aj 0, prázdny zoznam alebo reťazec
- false-výraz – vráti sa, ak je podmienka = **#f**
 - true-výraz a false-výraz nemusia byť rovnakého typu
 - Racket je dynamicky typovaný
- (**define** (test x) (**if** (> x 10) "Hurá" -7))
 - (test 20) $\rightsquigarrow$ "Hurá"
 - (test 0) $\rightsquigarrow$ -7

```
1 (define (test bmi)
2   (cond [(<= bmi 18.5) "anorexia?"]
3         [(<= bmi 25.0) "ideal :-")
4         [(<= bmi 30.0) "nadvaha"
5         [(<= bmi 40.0) "obezita"
6         [else "velryba?"]]))
```

- zakončiť **cond** špeciálnou formou **else** (alebo **#t**) nie je povinné, v tom prípade ale môže **cond** vrátiť **#<void>**
 - prítomnosť **#<void>** sa dá testovať pomocou **void?**
 - ak je výsledkom termu **#<void>** tak REPL nič nevypíše

```
1 (define (fact x)
2   (if (<= x 1)
3       1
4       (* x (fact (- x 1)))))
```

(fact 1000) $\rightsquigarrow$ 4023872600770937...000000000000000000

- Racket tiež štandardne pracuje s "bigint"

	Haskell	Racket
Prázdný zoznam	<code>[]</code>	<code>null</code>
Konštruktor	<code>(:)</code>	<code>cons</code>
Prvý prvok	<code>head</code>	<code>car</code>
Zvyšok zoznamu	<code>tail</code>	<code>cdr</code>
Test na prázdnoť	<code>null</code>	<code>null?</code>

- `(cons 1 (cons 2 null))` $\rightsquigarrow$ `'(1 2)`, to isté ako:
 - `(list 1 2)`
 - `'(1 2)` – o symbole `'` si povieme viac neskôr
- `null` je to isté ako `(list)` alebo `'()`
- `(cddar '(1 2 3) 4) =`
`(cdr (cdr (car '(1 2 3) 4)))) = '(3)`
- `car` – "Contents of the Address part of Register"
- `cdr` – "Cont. of the Decrement part of Reg." (čítaj "could-er")

- cons v skutočnosti vytvára dvojicu (cons cell):
 - `(cons 1 2)  $\rightsquigarrow$  '(1 . 2)`
 - `(cons (cons 1 2) (cons 3 null))  $\rightsquigarrow$  '((1 . 2) 3)`
 - nesprávny zoznam (improper list)
 - `(list? (cons 1 2))  $\rightsquigarrow$  #f`
 - `(pair? (cons 1 2))  $\rightsquigarrow$  #t`
- zoznamy sú iba vlastne špeciálny prípad vnorených dvojíc
 - končiacich null
 - `(cons (cons 1 null) (cons 3 null))  $\rightsquigarrow$  '((1) 3)`
 - správny zoznam (proper list)
 - `(list? (cons (cons 1 null) (cons 3 null))  $\rightsquigarrow$  #t`
 - `(pair? (cons (cons 1 null) (cons 3 null))  $\rightsquigarrow$  #t`
- car – vracia prvý člen dvojice
- cdr – vracia druhý člen dvojice

Zoznamy – sum

```
1 (define (sum xs)
2   (if (null? xs)
3       0
4       (+ (car xs) (sum (cdr xs)))))
5
6 (define (pokus x)
7   (sum (list 1 pokus "tri" 4 x)))
```

- (sum '(1 2 3 4)) $\rightsquigarrow$ 10
- (pokus 10) $\rightsquigarrow$ +: contract violation
 - kompilácia funkcie pokus – OK
 - typová kontrola ale prebehne až za behu

Zoznamy – append

```
1 (define (append xs ys)
2   (if (null? xs)
3       ys
4       (cons (car xs)
5             (append (cdr xs) ys))))
```

- `(append '(1 2) '("A" "BC"))` $\rightsquigarrow$ `'(1 2 "A" "BC")`

Zoznamy – map

```
1 (define (map f xs)
2   (if (null? xs)
3       null
4       (cons
5         (f (car xs))
6         (map f (cdr xs))))))
```

- `(map (lambda (x) (* 2 x)) '(1 2 3))  $\rightsquigarrow$  '(2 4 6)`

Dynamická kontrola typov

- nedokáže pri kompilácii odhaliť všetky chyby
 - (**define** (calc x) (x * 7))
 - (calc 2) $\rightsquigarrow$ application: not a procedure
 - (calc (**lambda** (f x) (f 10 x))) $\rightsquigarrow$ 70
- ale umožňuje vytvárať flexibilnejšie DŠ a programy

```
1 (define (sum xs)
2   (if (null? xs)
3       0
4       (if (number? (car xs))
5           (+ (car xs) (sum (cdr xs)))
6           (+ (sum (car xs))
7              (sum (cdr xs))))))
8
9 (define list
10  '(1 2 (3 (4 5) 6) 7 8 (9 ())))
```

- (sum list) $\rightsquigarrow$ 45
- (sum '(1 (2 "a") #t (3 ()))) $\rightsquigarrow$ car: contract

Dynamická kontrola typov

- nasledovná implementácia sum umožní vypočítať súčet aj pokiaľ zoznam obsahuje iné prvky ako čísla
 - tie považuje za rovné 0

```
1 (define (sum xs)
2   (cond [(null? xs) 0]
3         [(number? (car xs)) (+ (car xs) (sum (cdr xs)))]
4         [(list? (car xs)) (+ (sum (car xs)) (sum (cdr xs)))]
5         [else (sum (cdr xs))]))
```

(sum '(1 (2 "a") #t (3 ()))) $\rightsquigarrow$ 6

- vyhodnocuje priradenia vo vonkajšom kontexte, nezahŕňajúc aktuálne definované premenné
 - dá sa využiť na výmenu obsahu dvoch premenných
`(let ([x y] [y x]) (list x y))`

```

1 (let ([x 5])
2   (let ([x 2]
3         [y x]
4         #| [x 7] |# ] ;blokový komentár
5     (list x y)))

```

- po vyhodnotení vráti ' (2 5)
- po odstránení blokového komentára by sme dostali
 - let: duplicate identifier in: x
- nemôžeme sa odvolávať na neskôr definované premenné
 - `(let ([f (lambda (x) z)] [z 10]) (f 1))` $\rightsquigarrow$
 z: unbound identifier

- pracuje rovnako ako **let** ale po definovaní lokálnej premennej je viditeľná aj v nasledujúcich definíciách v **let***

```

1 (let* ([x 1]
2        [y (+ x 1)]
3        [x 7]) ;môžeme zatieniť
4        (list y x)) ;existujúcu premennú

```

- po vyhodnotení vráti ' (2 7), bez * by sme dostali:
 - (**let** ([x 1] [y (+ x 1)]) (**list** y x)) $\rightsquigarrow$
x: unbound identifier
- taktiež sa nemôžeme odvolávať na neskôr definované premenné
 - (**let*** ([f (**lambda** (x) z)] [z 10]) (f 1)) $\rightsquigarrow$
z: unbound identifier
- nie je až tak vhodný na výmenu premenných
 - (**let*** ([x y] [y x]) (**list** x y)) $\rightsquigarrow$ (y, y)
 - (**let*** ([t x] [x y] [y t]) (**list** x y)) $\rightsquigarrow$ (y, x)

- pracuje rovnako ako **let** ale môžeme sa odvolávať aj na neskoršie definované premenné (nemôžeme použiť ich obsah)
 - nutné pre definovanie vzájomne rekurzívnych funkcií

```
1 (letrec ([par? (lambda (n)
2             (or (zero? n)
3                 (nepar? (sub1 n)))))]
4   [nepar? (lambda (n)
5             (and (not (zero? n))
6                 (par? (sub1 n)))))]
7   (nepar? 11)) ;vracia #t
```

- nemôžeme opakovane definovať tú istú premennú:
 - (letrec ([x 10] [x 20]) x) $\rightsquigarrow$
letrec: duplicate identifier in: x
- vôbec nie je vhodný na výmenu premenných
 - (letrec ([x y] [y x]) (list x y)) aj
 - (letrec ([t x] [x y] [y t]) (list x y)) vráti:
y: undefined; cannot use before initialization

Meniteľné premenné – set!

```
1 (define a 10)           ; a je 10
2 (define f (lambda (x) (+ x a)))
3 (define b (+ 5 a))      ; b je 15
4 (set! a 7)              ; a zmutuje :-) na 7
5 (define c b)            ; c je 15
6 (define r1 (f 4))       ; r1 je 11
7 (define r2 (let ([a -2]) (f 4))) ; r2 je 11
```

- premenné pre uzáver (closure) funkcie f sa berú z prostredia (environment) existujúceho v čase jej definície
- doteraz sme mali nemenné premenné, preto hodnoty v uzávère nebolo možné zmeniť
- Racket umožňuje zmeniť hodnotu premennej, pokiaľ je stále viditeľná a preto aj na riadku 6 dostane výsledok 11 a nie 14
 - môžeme tomu zabrániť, zmenou definície na riadku 2:
 - (define f (let ([a a]) (lambda (x) (+ x a))))
- pri dynamickom scopingu by r2 bolo 2

mcons – meniteľné páry

- dvojice (cons cells) sú nemeniteľné (na rozdiel od jazyka Scheme, ktorý mal funkcie `set-car!` a `set-cdr!`)
 - môžeme zdieľať spoločné časti zoznamov
 - `list?` môže byť počítané počas vytvárania cons-bunky, tým pádom namiesto $O(n)$ má zložitosť $O(1)$
- pre meniteľné dvojice Racket používa `mcons`, `mcar`, `mcdr`, `mpair?`, `set-mcar!` a `set-mcdr!`
 - (`define` `mpair` (`mcons` 1 (`mcons` #t "a"))))
 - (`mcdr` `mpair`) $\rightsquigarrow$ (`mcons` #t "a")
 - (`set-mcar!` (`mcdr` `mpair`) "boo")
 - `mpair` $\rightsquigarrow$ (`mcons` 1 (`mcons` "boo" "a")))
- `mcar` a spol. sa nedajú použiť na cons-bunku
- `car`, ... (vrátane `length` atď.) sa nedajú použiť na mcons-bunku

Lenivé vyhodnocovanie

Odložené vyhodnotenie

- sémantika jazyka špecifikuje pre každý jazykový konštrukt, kedy sa jeho podvýrazy vyhodnocujú
 - Racket, C, Java má striktné vyhodnocovanie argumentov
 - strict, eager, call-by-value
 - ale napr. vetvy v if sa vyhodnocujú až podľa podmienky
 - to je veľmi dôležité

```
1 (define (bad-if p t e) (if p t e))
2 (define (fact n)
3   (bad-if (< n 1)
4           1
5           (* n (fact (sub1 n)))))
```

- všetky volania fact, aj (fact 0) sa zacyklia
 - lebo pred zavolaním bad-if sa vyhodnotia všetky argumenty
- výmena bad-if za if funkciu fact opravi

```
1 (define (ok-if p t e) (if p (t) (e)))
2 (define (fact n)
3   (ok-if
4     (< n 1)
5     (lambda () 1)
6     (lambda () (* n (fact (sub1 n)))))
7   ))
```

- vieme, že funkcie nevyhodnocujú svoje telo až do zavolania
 - preto ok-if namiesto hodnoty pre t a e vetvu berie funkciu bez argumentov, ktorej telom je pôvodný argument
 - tým sa zabráni vyhodnoteniu argumentov
 - v tele if sa potom vyhodnotí t alebo f (t.j. (t) alebo (e))
- $(\text{lambda } () e)$ je **thunk** pre e
 - thunk sa ako argument nevyhodnocuje, už je hodnotou
 - vďaka uzáverom môžeme thunk spraviť kdekoľvek a následne ho kedykoľvek vyhodnotiť

Thunks – zhrnutie

- e – vyhodnotí výraz e na nejakú hodnotu
 - $(+ \ 1 \ 2)$ sa hneď vyhodnotí na 3
- $(\text{lambda } () \ e)$
bezargumentová funkcia pre **thunking** výrazu e
 - $(\text{lambda } () \ (+ \ 1 \ 2))$ sa vyhodnotí na funkciu
(telo sa nepočíta)
- (et) – zavolá bezargumentovú funkciu et (**think**) a vyhodnotí jej telo, teda výraz e z $(\text{lambda } () \ e)$
 - $((\text{lambda } () \ (+ \ 1 \ 2)))$ sa vyhodnotí na 3

Zbytočné (opakované) vyhodnocovanie thunk-ov

```
1 (define (slow x) (begin (sleep 2) x))
2 (define (mult x y-thunk)
3   (cond [(= x 0) 0]
4         [(= x 1) (y-thunk)]
5         [else (+ (y-thunk)
6                  (mult (sub1 x)
7                        y-thunk))]))
8 (define (mul10 x)
9   (mult x (lambda () (slow 10))))
```

- **begin** – vyhodnotí postupne všetky svoje argumenty (zľava doprava) a vráti hodnotu posledného
 - má zmysel len pokiaľ sa udejú nejaké vedľajšie efekty
- (mul10 0) – nevoláme žiadne slow (0 s)
- (mul10 1) – vyhodnocujeme jedno slow (2 s)
- (mul10 5) – vyhodnocuje slow opakované 5-krát (10 s)

Lenivé vyhodnocovanie (lazy evaluation)

Ak drahý výpočet nemá vedľajšie efekty, tak by sme chceli:

- nepočítať ho pokiaľ ho nebude treba,
- výsledok odložiť pre ďalšie použitia.

Haskell je príkladom takéhoto riešenia.

Ukážeme, ako lazy evaluation implementovať v Racket pomocou:

- thunks,
- mcons.

Lenivé vyhodnocovanie v Racket

```
1 (define (dly th) (mcons #f th)) ; dly ako delay
2 (define (frc pr) ; frc ako force
3   (cond [(mcar pr) (mcdr pr)]
4         [else (begin (set-mcar! pr #t)
5                       (set-mcdr! pr ((mcdr pr))
6                                     (mcdr pr)))]])
7 (define (slow x) (begin (sleep 2) x))
8 (define (mult x y-pr)
9   (cond [(= x 0) 0]
10         [(= x 1) (frc y-pr)]
11         [else (+ (frc y-pr)
12                  (mult (sub1 x) y-pr))]))
13 (define (mul10 x) (mult x (dly (lambda () (slow 10))))))
```

- (mcons #f th) – th je ešte nevyhodnotený thunk
- (mcons #t val) – val je už raz vypočítaná hodnota pre thunk (ten sme potom zahodili a nahradili val)
- (mul10 0) – nevyhodnocujeme slow ani raz (0 s)
- (mul10 1) – vyhodnotíme slow raz (2 s)
- (mul10 5) – slow sa používa 5×, ale vyhodnotí sa iba 1× (2 s)

Makrá

- **definícia makra** popisuje ako transformovať novú syntax do syntaxe v pôvodnom jazyku
 - umožňuje definovať vlastný syntaktický cukor
- **makro-systém** je jazyk pre definíciu makier
 - môže to byť podjazyk iného väčšieho jazyka
 - môže byť s ním úzko prepojený (tokenizácia, scoping)
- **rozbalenie makier** (macro expansion) je prepísanie každého **použitia makra** do syntaxe pôvodného jazyka
 - predtým než sa program vôbec začne kompilovať

- definícia makra m zavedie novú špeciálnu formu m
 - $(m \dots)$ sa rozbalí podľa definície makra

Príklady

- $(\text{my-if } p \text{ then } t \text{ **else** } e) \rightsquigarrow (\text{if } p \text{ } t \text{ } e)$
 - slová `then` a `else` sa stanú povinnou súčasťou `my-if`
- $(\text{replace } e_1 \ e_2) \rightsquigarrow e_2$
 - vyhodí e_1 vloží e_2 (pričom e_1 ostane v zdrojovom kóde)
 - bez makier by sme museli vyhodnotiť aj e_1
- $(\text{delay } e) \rightsquigarrow (\text{dly } (\text{lambda } () \ e))$
 - pri `dly` sme museli ručne zabaliť argument do thunk-u
 - toto makro to spraví automaticky
 - bez makier by sme museli vyhodnotiť e

Problémy s makrami – tokenizácia a uzátvorkovanie

- tokenization – chceme, aby makrá nepracovali na znakoch ale tokenoch, aby sme sa vyhli falošnej detekcii použitia makra
 - napríklad: nech sa makro head rozbalí na car
 - nechceme prepísať premennú head-less na premennú car-less
 - ale v jazyku C by sme to chceli (ide o odčítanie dvoch premenných)
- uzátvorkovanie – majme C makro `#define MUL(x,y) x*y`
 - `MUL(1+2,3+4)*5` znamená $1 + 2 * 3 + 4 * 5 = 27$ a nie 50
 - preto C makrá plne zátvorkujeme: `((x)*(y))`
 - Racket nemá tento problém vďaka svojej plne uzátvorkovanej syntaxi
 - `(macro ...)` sa rozbalí na (čosi v tých istých zátvorkách)

Makrá a lokálne premenné – ďalšie problémy

- (**let** ([head 1] [car 2]) head) sa naivne rozbalí na:
(**let** ([car 1] [car 2]) car) – duplicitná definícia x
- (**let*** ([head 1] [car 2]) head) sa naivne rozbalí na:
(**let*** ([car 1] [car 2]) car) – vráti 2, čakali sme 1
- problém je v tom, že lokálne premenné nezatieňujú makrá
 - preto C má konvenciu:
názvy makier sú iba z veľkých písmen, ostatné názvy
(premenných, funkcií, ...) obsahujú aj malé písmeno
- Racket nemá tento problém
 - lokálne premenné zatieňujú makrá

Syntax makier v jazyku Racket

```
1 (define-syntax my-if
2   (syntax-rules (then else)
3     [(my-if p then et else ef) (if p et ef)]
4     [(my-if p then et) (if p et (void))]))
5
6 (my-if #t then 10 else (car null)) ; 10
7 (my-if #f then 10)                 ; #<void>
8
9 (define-syntax replace
10   (syntax-rules ()
11     [(replace co cim) cim]))
12
13 (replace (error "Fiha") 7) ; 7
14
15 (define-syntax delay
16   (syntax-rules ()
17     [(delay e) (mcons #f (lambda () e))]))
18
19 (delay (begin (print "Pocitam...") 42))
20 ; (mcons #f #<procedure>)
```

Ďalšie makro problémy

- medzi nasledovnými funkciami nie je pre používateľa rozdiel:
 - (**define** (dbl x) (+ x x))
 - (**define** (dbl x) (* 2 x))
- ale v nasledovných makrách je pozorovateľný rozdiel:

```
1 (define-syntax dbl1
2   (syntax-rules ()
3     [(dbl x) (+ x x)]))
4
5 (dbl1 (begin (print "Hi") 42))
6 ; "Hi""Hi"84 - vyhodnotí x dva krát
```

```
1 (define-syntax dbl2
2   (syntax-rules ()
3     [(dbl x) (* 2 x)]))
4
5 (dbl2 (begin (print "Hi") 42))
6 ; "Hi"84 - vyhodnotí x iba raz
```

Riešenie pomocou lokálnych premenných

- nasledovné riešenie vyhodnotí x iba raz

```
1 (define-syntax dbl
2   (syntax-rules ()
3     [(dbl x)
4       (let ([tmp x])
5         (+ tmp tmp))]))
6 (dbl (begin (print "Hi") 42))
7 ; "Hi"84
```

- nasledovné makro nevyhodnocuje argumenty zľava doprava
 - opäť riešiteľné pomocou lokálnych premenných

```
1 (define-syntax take
2   (syntax-rules (from)
3     [(take co from coho) (- coho co)]))
4 (define-syntax trace
5   (syntax-rules ()
6     [(trace x) (let ([tmp x]) (begin (print tmp) tmp))]))
7 (take (trace 2) from (trace 6))
8 ; 624 a nie 2 6 4 (podľa poradia argumentov)
```

Problémy s lokálnymi premennými v makrách

```
1 (define-syntax dbl
2   (syntax-rules ()
3     [(dbl x)
4       (let ([y 1]) (* 2 x y))]))
```

- naivné rozbalenie (**let** ([y 7]) (dbl y)):
(**let** ([y 7]) (**let** ([y 1]) (* 2 y y))) $\rightsquigarrow$ 2
 - týmto problémom trpí napríklad makro-systém C
- Racket, keď je to potrebné, nahrádza lokálne premenné v makrách novými unikátnymi premennými
 - 1. súčasť makro-hygiény v jazyku Racket

Problém s nelokálnymi premennými v makrách

```
1 (define-syntax dbl  
2   (syntax-rules ()  
3     [(dbl x) (* 2 x)]))
```

- naivné rozbalenie (**let** ([* +]) (dbl 42)):
(**let** ([* +]) (* 2 42)) $\rightsquigarrow$ 44 a nie 84
 - týmto problémom trpí C
 - dá sa použiť na simulovanie dynamického scopingu v C
- Racket vyhľadáva hodnoty premenných lexikálne podľa miesta, kde boli definované a nie kde sú použité
 - 2. súčasť makro-hygiény v jazyku Racket